SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE	DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Program Name: B. Tech	**Assignment Type: Lab**	**Academic Year:**2025-2026
Course Coordinator Name	Dr. Rishabh Mittal	
Instructor(s) Name	Mr. S Naresh Kumar Ms. B. Swathi Dr. Sasanko Shekhar Gantayat Mr. Md Sallauddin Dr. Mathivanan Mr. Y Srikanth Ms. N Shilpa Dr. Rishabh Mittal (Coordinator) Dr. R. Prashant Kumar Mr. Ankushavali MD Mr. B Viswanath Ms. Sujitha Reddy Ms. A. Anitha Ms. M.Madhuri Ms. Katherashala Swetha Ms. Velpula sumalatha Mr. Bingi Raju	

CourseCode	23CS002PC304	**Course Title**	AI Assisted Coding
Year/Sem	III/II	**Regulation**	R23
Date and Day of Assignment	**Week6 – Wednusday**	**Time(s)**	23CSBTB01 To 23CSBTB52
Name:	D.udayan	**Holl.no**	2303A54017

Assignment Number:12.3(Present assignment number)/**24**(Total number of assignments)

Q.No.	Question	*Expected Time to complete*
1	**Lab 12: Algorithms with AI Assistance Sorting, Searching, and Algorithm Optimization Using AI Tools** **Lab Objectives** The objectives of this laboratory exercise are to:	Week3 - Wednesday

- Apply AI-assisted programming techniques to implement sorting and searching algorithms.
- Analyze and compare algorithm efficiency using time and space complexity.
- Understand how AI tools can suggest optimizations and alternative algorithmic approaches.
- Strengthen problem-solving skills through real-world, data-driven scenarios.

Learning Outcomes

After completing this lab, students will be able to:

- Implement and optimize classic algorithms using AI-assisted coding tools.
- Compare multiple algorithms for the same problem and justify their selection.
- Measure and analyze runtime performance using experimental data.
- Critically review and refine AI-generated algorithmic solutions.

Task 1: Sorting Student Records for Placement Drive

Scenario

SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student records must be sorted by **CGPA in descending order**.

Tasks

1. Use **GitHub Copilot** to generate a program that stores student records (Name, Roll Number, CGPA).
2. Implement the following sorting algorithms using AI assistance:
   - Quick Sort
   - Merge Sort
3. Measure and compare runtime performance for large datasets.
4. Write a function to display the **top 10 students** based on CGPA.

Expected Outcome

- Correctly sorted student records.
- Performance comparison between Quick Sort and Merge Sort.
- Clear output of top-performing students.

```python
#Sorting Student Records for Placement Drive
#SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student records must be sorted by CGPA in descending order.
#Use GitHub Copilot to generate a program that stores student records
# (Name, Roll Number, CGPA).
# 2. Implement the following sorting algorithms using AI assistance:
# o Quick Sort
# o Merge Sort
# 3. Measure and compare runtime performance for large datasets.
# 4. Write a function to display the top 10 students based on CGPA. give an example also
import random
import time
class Student:
    def __init__(self, name, roll, cgpa):
        self.name = name
        self.roll = roll
        self.cgpa = cgpa

    def __repr__(self):
        return f"{self.name} | Roll: {self.roll} | CGPA: {self.cgpa}"
def quick_sort(students):
    if len(students) <= 1:
        return students

    pivot = students[len(students)//2].cgpa

    left = [s for s in students if s.cgpa > pivot]
    middle = [s for s in students if s.cgpa == pivot]
    right = [s for s in students if s.cgpa < pivot]

    return quick_sort(left) + middle + quick_sort(right)
def merge_sort(students):
    if len(students) <= 1:
        return students

    mid = len(students)//2
    left = merge_sort(students[:mid])
    right = merge_sort(students[mid:])

    return merge(left, right)


def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i].cgpa >= right[j].cgpa:
            result.append(left[i])
            i += 1
```

ai_ass.12.3 > task.01.py > ...

```python
def merge(left, right):
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result
def display_top_10(students):
    print("\n🏆 Top 10 Students (by CGPA):")
    for i, student in enumerate(students[:10], start=1):
        print(f"{i}. {student}")
def generate_students(n):
    students = []
    for i in range(n):
        name = f"Student{i+1}"
        roll = 1000 + i
        cgpa = round(random.uniform(5.0, 10.0), 2)
        students.append(Student(name, roll, cgpa))
    return students
if __name__ == "__main__":

    # Example dataset size
    N = 5000

    students = generate_students(N)

    # ----- Quick Sort Timing -----
    start = time.time()
    quick_sorted = quick_sort(students)
    quick_time = time.time() - start

    # ----- Merge Sort Timing -----
    start = time.time()
    merge_sorted = merge_sort(students)
    merge_time = time.time() - start

    # Runtime comparison
    print("\n⏱ Runtime Comparison")
    print(f"Quick Sort Time : {quick_time:.5f} seconds")
    print(f"Merge Sort Time : {merge_time:.5f} seconds")

    # Display Top Students
    display_top_10(quick_sorted)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\MYPC\OneDrive\Desktop\ai assisted coding> & C:\Users\MYP
4. Student384 | Roll: 1383 | CGPA: 9.99
5. Student605 | Roll: 1604 | CGPA: 9.99
6. Student1177 | Roll: 2176 | CGPA: 9.99
7. Student1347 | Roll: 2346 | CGPA: 9.99
8. Student1372 | Roll: 2371 | CGPA: 9.99
9. Student1702 | Roll: 2701 | CGPA: 9.99
10. Student1937 | Roll: 2936 | CGPA: 9.99
PS C:\Users\MYPC\OneDrive\Desktop\ai assisted coding>
```

Task 2: Implementing Bubble Sort with AI Comments

- **Task**: Write a Python implementation of **Bubble Sort**.
- **Instructions**:
  - Students implement Bubble Sort normally.
  - Ask AI to generate **inline comments explaining key logic** (like swapping, passes, and termination).
  - Request AI to provide **time complexity analysis**.
- **Expected Output**:
  - A Bubble Sort implementation with AI-generated explanatory comments and complexity analysis.

ai_ass.12.3 > task.02.py > ...

```python
# Implementing Bubble Sort with AI Comments
# • Task: Write a Python implementation of Bubble Sort.
# • Instructions:
# • Students implement Bubble Sort normally.
# • Ask AI to generate inline comments explaining key logic (like
# swapping, passes, and termination).
# • Request AI to provide time complexity analysis.

def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        swapped = False

        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True

        if not swapped:
            break

    return arr


numbers = [64, 34, 25, 12, 22, 11, 90]

print("Original List:", numbers)

sorted_numbers = bubble_sort(numbers)

print("Sorted List:", sorted_numbers)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\MYPC\OneDrive\Desktop\ai assisted coding> & C:\Users\MYPC\AppData\Local\Progra
7. Student1347 | Roll: 2346 | CGPA: 9.99
8. Student1372 | Roll: 2371 | CGPA: 9.99
9. Student1702 | Roll: 2701 | CGPA: 9.99
10. Student1937 | Roll: 2936 | CGPA: 9.99
PS C:\Users\MYPC\OneDrive\Desktop\ai assisted coding> & C:\Users\MYPC\AppData\Local\Progra
Original List: [64, 34, 25, 12, 22, 11, 90]
Sorted List: [11, 12, 22, 25, 34, 64, 90]
```

Task 3: Quick Sort and Merge Sort Comparison

- **Task:** Implement Quick Sort and Merge Sort using recursion**.**
- **Instructions:**
  - Provide AI with partially completed functions for recursion.
  - Ask AI to complete the missing logic and add docstrings.
  - Compare both algorithms on random, sorted, and reverse-sorted lists.
- **Expected Output:**
  - Working Quick Sort and Merge Sort implementations.
  - AI-generated explanation of average, best, and worst-case complexities.

ai_ass.12.3 > task.03.py > ...

```python
# Quick Sort and Merge Sort Comparison
# • Task: Implement Quick Sort and Merge Sort using recursion.
# • Instructions:
# • Provide AI with partially completed functions for recursion.
# • Ask AI to complete the missing logic and add docstrings.
# • Compare both algorithms on random, sorted, and reverse-sorted
# lists.

import random
import time


def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[len(arr) // 2]

    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]

    return quick_sort(left) + middle + quick_sort(right)


def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2

    left_half = merge_sort(arr[:mid])
    right_half = merge_sort(arr[mid:])

    return merge(left_half, right_half)


def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

```

ai_ass.12.3 > task.03.py > ...

```python
def merge(left, right):

    result.extend(left[i:])
    result.extend(right[j:])
    return result


def test_algorithm(sort_function, data):
    start = time.time()
    sort_function(data.copy())
    return time.time() - start


size = 5000

random_list = [random.randint(1, 10000) for _ in range(size)]
sorted_list = sorted(random_list)
reverse_sorted_list = sorted(random_list, reverse=True)

print("\n--- Performance Comparison ---")

datasets = {
    "Random List": random_list,
    "Sorted List": sorted_list,
    "Reverse Sorted List": reverse_sorted_list
}

for name, dataset in datasets.items():
    quick_time = test_algorithm(quick_sort, dataset)
    merge_time = test_algorithm(merge_sort, dataset)

    print(f"\n{name}")
    print(f"Quick Sort Time : {quick_time:.5f} seconds")
    print(f"Merge Sort Time : {merge_time:.5f} seconds")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\MYPC\OneDrive\Desktop\ai assisted coding> & C:\Users\MYPC\AppDat
Sorted List
Quick Sort Time : 0.00479 seconds
Merge Sort Time : 0.00438 seconds

Reverse Sorted List
Quick Sort Time : 0.00379 seconds
Merge Sort Time : 0.00477 seconds
```

Task 4 (Real-Time Application – Inventory Management System)

Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.
2. Sort products by price or quantity for stock analysis.

Task:

- Use AI to suggest the most efficient search and sort algorithms for this use case.
- Implement the recommended algorithms in Python.
- Justify the choice based on dataset size, update frequency, and performance requirements.

Expected Output:

- A table mapping operation → recommended algorithm → justification.
- Working Python functions for searching and sorting the inventory.

```python
# Real-Time Application – Inventory Management System)
# Scenario: A retail store’s inventory system contains thousands of products,
# each with attributes like product ID, name, price, and stock quantity. Store staff
# need to:
# 1. Quickly search for a product by ID or name.
# 2. Sort products by price or quantity for stock analysis.
# Task:
# • Use AI to suggest the most efficient search and sort algorithms for this
# use case.
# • Implement the recommended algorithms in Python.
# • Justify the choice based on dataset size, update frequency, and
# performance requirements.
class Product:
    def __init__(self, pid, name, price, quantity):
        self.pid = pid
        self.name = name
        self.price = price
        self.quantity = quantity

    def __repr__(self):
        return f"{self.pid} | {self.name} | Price:{self.price} | Qty:{self.quantity}"


class Inventory:
    def __init__(self):
        self.products = []
        self.id_index = {}
        self.name_index = {}

    def add_product(self, product):
        self.products.append(product)
        self.id_index[product.pid] = product
        self.name_index[product.name.lower()] = product

    def search_by_id(self, pid):
        return self.id_index.get(pid, "Product not found")

    def search_by_name(self, name):
        return self.name_index.get(name.lower(), "Product not found")

    def sort_by_price(self):
        return sorted(self.products, key=lambda x: x.price)

    def sort_by_quantity(self):
        return sorted(self.products, key=lambda x: x.quantity)


inventory = Inventory()
```

ai_ass.12.3 > task.04.py > ...

```python

inventory = Inventory()

inventory.add_product(Product(101, "Laptop", 75000, 15))
inventory.add_product(Product(102, "Mouse", 500, 120))
inventory.add_product(Product(103, "Keyboard", 1500, 60))
inventory.add_product(Product(104, "Monitor", 12000, 25))
inventory.add_product(Product(105, "Headphones", 2500, 40))

print("\nSearch by ID:")
print(inventory.search_by_id(103))

print("\nSearch by Name:")
print(inventory.search_by_name("mouse"))

print("\nSorted by Price:")
for p in inventory.sort_by_price():
    print(p)

print("\nSorted by Quantity:")
for p in inventory.sort_by_quantity():
    print(p)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\MYPC\OneDrive\Desktop\ai assisted coding> & C:\Users\MYPC\

Sorted by Quantity:
101 | Laptop | Price:75000 | Qty:15
104 | Monitor | Price:12000 | Qty:25
105 | Headphones | Price:2500 | Qty:40
103 | Keyboard | Price:1500 | Qty:60
102 | Mouse | Price:500 | Qty:120
```

Task 5: Real-Time Stock Data Sorting & Searching
Scenario:

An AI-powered **FinTech Lab** at SR University is building a tool for analyzing **stock price movements**. The requirement is to quickly **sort stocks by daily gain/loss** and search for specific stock symbols efficiently.

- Use **GitHub Copilot** to fetch or simulate stock price data (Stock Symbol, Opening Price, Closing Price).
- Implement sorting algorithms to rank stocks by **percentage change**.
- Implement a **search function** that retrieves stock data instantly when a stock symbol is entered.
- Optimize sorting with **Heap Sort** and searching with **Hash Maps**.
- Compare performance with standard library functions (sorted(), dict lookups) and analyze trade-offs.

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.

ai_ass.12.3 > task.05.py > ...

```python
# Real-Time Stock Data Sorting & Searching
# Scenario:
# An AI-powered FinTech Lab at SR University is building a tool for analyzing
# stock price movements. The requirement is to quickly sort stocks by daily
# gain/loss and search for specific stock symbols efficiently.
# • Use GitHub Copilot to fetch or simulate stock price data (Stock
# Symbol, Opening Price, Closing Price).
# • Implement sorting algorithms to rank stocks by percentage change.
# • Implement a search function that retrieves stock data instantly when a
# stock symbol is entered.
# • Optimize sorting with Heap Sort and searching with Hash Maps.
# • Compare performance with standard library functions (sorted(), dict
# lookups) and analyze trade-offs.
# Note: Report should be submitted as a word document for all tasks in a single
# document with prompts, comments & code explanation, and output and if
# required, screenshots.

import random
import time


class Stock:
    def __init__(self, symbol, open_price, close_price):
        self.symbol = symbol
        self.open_price = open_price
        self.close_price = close_price
        self.change = ((close_price - open_price) / open_price) * 100

    def __repr__(self):
        return f"{self.symbol} | Open:{self.open_price:.2f} | Close:{self.close_price:.2f} | Change:{self.change:.2f}%"


def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2

    if left < n and arr[left].change > arr[largest].change:
        largest = left

    if right < n and arr[right].change > arr[largest].change:
        largest = right

    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)
```

ai_ass.12.3 > task.05.py > ...

```python
def heap_sort(arr):
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)

    for i in range(n - 1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i]
        heapify(arr, i, 0)

    return arr[::-1]


def generate_stocks(n):
    stocks = []
    for i in range(n):
        symbol = f"STK{i}"
        open_price = random.uniform(100, 1000)
        close_price = open_price + random.uniform(-50, 50)
        stocks.append(Stock(symbol, open_price, close_price))
    return stocks


def build_hash_map(stocks):
    return {stock.symbol: stock for stock in stocks}


def search_stock(stock_map, symbol):
    return stock_map.get(symbol, "Stock not found")


stocks = generate_stocks(1000)

start = time.time()
heap_sorted = heap_sort(stocks.copy())
heap_time = time.time() - start

start = time.time()
builtin_sorted = sorted(stocks, key=lambda x: x.change, reverse=True)
builtin_time = time.time() - start

stock_map = build_hash_map(stocks)

print("Heap Sort Time:", heap_time)
print("Built-in Sort Time:", builtin_time)

print("\nSearch Result:")
print(search_stock(stock_map, "STK10"))

print("\nTop 5 Stocks by Percentage Change:")
for stock in heap_sorted[:5]:
    print(stock)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\MYPC\OneDrive\Desktop\ai assisted coding> & C:\Users\MYPC\AppData\Lo

Top 5 Stocks by Percentage Change:
STK386 | Open:101.12 | Close:150.37 | Change:48.70%
STK667 | Open:102.67 | Close:150.80 | Change:46.88%
STK73 | Open:107.07 | Close:154.30 | Change:44.11%
STK722 | Open:109.05 | Close:155.73 | Change:42.81%
STK555 | Open:111.89 | Close:158.20 | Change:41.39%
PS C:\Users\MYPC\OneDrive\Desktop\ai assisted coding>
```